TP1: Module GNSS

ITI3: Capteurs

Robin Condat, Antoine Vaugeois

1 Objectifs de la séance

Aujourd'hui, on découvre et on manipule un capteur GNSS. Au programme, on va réceptionner du signal satellite, on va sortir faire des acquisitions à l'extérieur et calculer des distances en vol d'oiseau.

1.1 Matériel nécessaire

- Une Raspberry Pi 4
- Un module GNSS DFRobot TEL0157 (lien documentation)
- Un afficheur LCD 1602
- Un parapluie ou k-way (s'il pleut, pour les acquisitions...)

1.2 Packages Python utilisés

- *smbus*: Pour communiquer avec des périphériques I2C
- *folium*: Pour tracer des coordonnées GNSS sur une carte OpenStreetMap
- *RPLCD*: Pour afficher sur le module LCD
- *csv*: Pour construire des beaux tableaux (utile ici pour l'enregistrement des acquisitions dans un fichier CSV)

2 Mise en place

- Connectez en I2C le module GNSS TEL0157 et l'afficheur LCD 1602 à la Raspberry Pi. N'hésitez pas à faire vérifier vos branchements si vous n'êtes pas sûr !
- Mettez sous tension la Raspberry Pi et vérifiez que le module GNSS reçoit des signaux satellites. La réception dépend beaucoup de l'environnement. Si vous ne recevez pas de signal, déplacez votre module proche d'une fenêtre de la salle.

- Activez l'environnement virtuel python `env_capteurs`.
- Téléchargez le code disponible sur Moodle, décompressez l'archive zip et renseignez le chemin de votre dossier dans la variable d'environnement `PYTHONPATH` :

```
export PYTHONPATH=/chemin/vers/votre/dossier/TP:$PYTHONPATH
```

- Détectez les adresses des périphériques connectés en I2C via la commande suivante:

```
i2cdetect -y 1
```

- Complétez le fichier `config/constants.py` avec les adresses trouvées.

3 Lecture de trames NMEA

Les trames NMEA (National Marine Electronics Association) sont des messages standardisés utilisés par les récepteurs GNSS (Global Navigation Satellite System) pour transmettre des informations de position, de temps, de vitesse et bien plus encore. Ces trames, généralement envoyées sous forme de texte ASCII via un port série, permettent de communiquer les données reçues des satellites aux systèmes ou appareils connectés.

Chaque trame commence par un symbole $, suivi d'un identifiant unique qui indique le type de données transmises. Les champs suivants contiennent des informations spécifiques, séparées par des virgules. Enfin, une somme de contrôle (checksum) assure l'intégrité des données.

- Exécutez le programme `script/get_nmea_frames.py`.
- Identifiez les provenances des signaux (de quel système de satellite provient le signal ? Comment fait-on pour identifier cela ?)
- Concentrez vous sur une trame GGA. Quelles informations pouvez-vous extraire de cette trame ? Idem pour une trame RMC.
- Exécutez le même programme, mais pour obtenir uniquement les trames ne provenant que de la constellation GPS, puis BEIDOU, puis GLONASS. Que constatez-vous ?

4 Coordonnées géographiques

Il existe plusieurs formats de coordonnées géographiques, chacun adapté à des contextes spécifiques en fonction de la compression de l'information nécessaire et de la précision souhaitée. On va voir ici 3 formats assez répandus.

Le premier, le plus connu, est le format **Degrés Décimaux (DD)**. C'est le format utilisé notamment dans les systèmes de navigation que vous utilisez au quotidien. Ce format utilise uniquement des nombres décimaux pour représenter les degrés. Les latitudes Nord et les longitudes Est sont positives, tandis que les latitudes Sud et longitudes Ouest sont négatives.

```
Exemple:  48.5631, -16.7963
```

Dans le contexte de la cartographie et la navigation, le format **Degrés, Minutes et Secondes (DMS)** est souvent préféré car plus précis. Les coordonnées sont divisées en degrés (°), minutes (’), et secondes (”). Dans ce contexte, un degré est égal à 60 minutes, et une minute est égale à 60 secondes. Les directions cardinales indiquent la position : N ou S pour la latitude, E ou W pour la longitude.

```
Exemple:  63° 41’ 32" N, 28° 46’ 2" E
```

Enfin, le format **Degrés et Minutes Décimales (DMM)** est un compromis entre les deux premiers formats: il est plus précis que le DMS, mais moins compact que le DD. Les coordonnées sont exprimées ainsi en degrés entiers et en minutes avec une partie décimale. Les directions cardinales indiquent la position, comme pour le format DMS.

```
Exemple:  52° 14.8496’ N, 2° 21.1344’ E
```

- Quelle est la distance parcouru lorsqu'on se déplace sur la surface de la Terre d'un degré ? D'une minute ? D'une seconde ?
- Quel est le format des coordonnées GNSS lues avec le module ? Avec quelle précision (en mètres) ?
- Dans le fichier `sensor/GNSS/gnss.py`, complétez la fonction `update_coordinate` de la classe `struct_GNSS_coordinate` pour convertir les coordonnées dans les 3 formats précédemment cités. Le format DD est représenté par un float, tandis que les deux autres formats sont représentés par une chaine de caractères.
- Modifiez le code `core/gnss_position.py` pour récupérer les coordonnées dans un des formats et les afficher sur le LCD.
- Testez votre programme en déplacant le capteur GNSS à l'extérieur.

5 Acquisitions

Il est temps de faire votre première acquisition de coordonnées GNSS !

- Exécutez le programme `scripts/get_position.py` en mettant en argument le nom d'un fichier CSV pour l'enregistrement des positions au fil du temps.
- Faites un tour dans l'enceinte de l'INSA avec votre planche d'essai dans votre boite fermée. Privilégiez les espaces extrérieurs pour une meilleure réception et donc une meilleure précision. Enfin, évitez de vous presser, car ça rendra vos mesures de position très peu précises. Bon voyage :D
- Exécutez le programme `postprocessing/plot_gnss.py` pour tracer le trajet que vous venez d'effectuer. Que constatez-vous ?

6 Calcul de distance

Maintenant qu'on est capable de récupérer une position GNSS, on peut calculer des distances par rapport à un point d'intérêt. La distance calculée sera en "vol d'oiseau", c'est à dire sans prendre en compte les routes possibles.

La distance angulaire entre deux points A et B, de latitudes ϕ_A et ϕ_B, de longitudes λ_A et λ_B, est:

$$D_{A,B} = \arccos\left(\sin(\phi_A) \cdot \sin(\phi_B) + \cos(\phi_A) \cdot \cos(\phi_B) \cdot \cos(\lambda_A - \lambda_B)\right) \quad (1)$$

- Complétez dans le code `core/gnss_distance.py` la fonction `compute_distance` pour calculer la distance entre deux points en coordonnées DD (tuple latitude et longitude)
- Complétez la fonction `gnss_distance` pour récupérer la distance entre les coordonnées GNSS du capteur et un point d'intérêt (en entrée de la fonction)
- Testez le programme `scripts/get_distance.py` avec les coordonnées suivantes:
  - New York: (40.712784,-74.005941)
  - Sydney: (-33.867487, 151.206990)
  - Paris: (48.856614, 2.352222)
  - Rouen: (49.443232, 1.099971)
  - Entrée du Batiment Darwin: (49.384330, 1.071885)
- Entrez les coordonnées de l'arche à côté du batiment Dumont D'Urville (49.384210, 1.069372) et déplacez vous jusqu'à ce point d'intérêt avec votre capteur GNSS. La précision est-elle au rendez-vous ?

7 CHASSE AU TRESOR !!!

Si vous en êtes ici, mes félicitations ! (en espérant que vous avez bien répondu à TOUTES les questions...)

Avant de finir ce TP, j'ai caché un post-it dans l'enceinte de l'INSA. Dès que vous êtes prêts, je lancerai votre programme avec mes coordonnées secrètes. Retrouvez-le, et prenez un selfie avec que vous pourrez intégrer à votre rendu.

S'il vous plait, n'enlevez pas le post-it trouvé pour les autres binomes...